

CPU SCHEDULING

operating system

CPU SCHEDULING

CONTENTS

01

CPU 스케줄링

02

스케줄러의 계층적 구조

03

스케줄링의 고려 사항

04

선점형 vs 비선점형

05

스케줄링 평가 기준

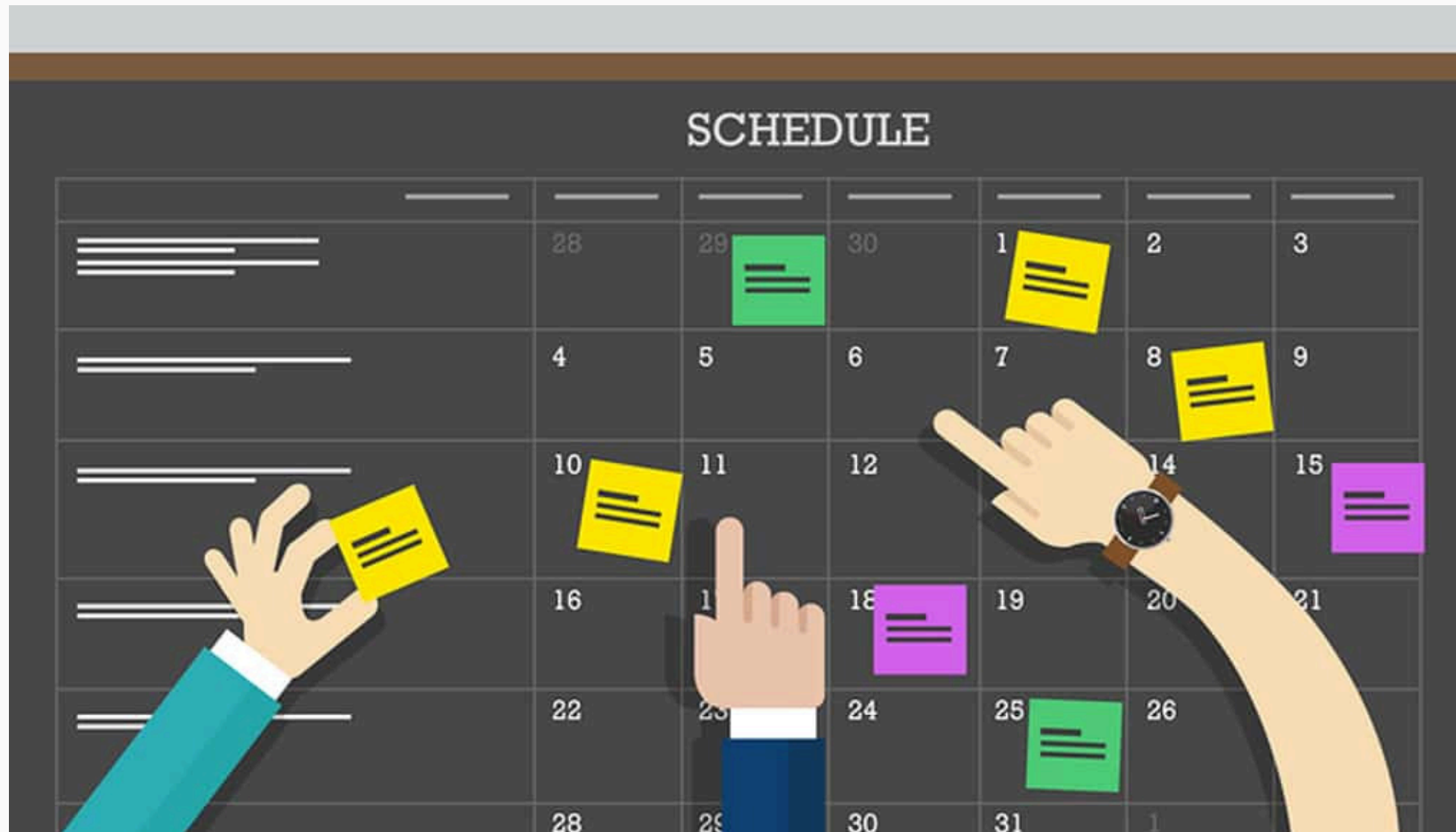
06

스케줄링 알고리즘

01

CPU 스케줄링이란?

CPU 스케줄링이란?



- 시스템의 한정된 자원을 사용하려는 여러 경쟁자 중에서 특정 시점에 어떤 대상에게 자원을 할당할지 결정하는 정책 및 행위

CPU 스케줄링의 목적

공평성

모든 프로세스가 자원을 공평하게 배정받아야 함

효율성

시스템 자원이 유휴 시간 없이 사용되어야 함

안정성

중요 프로세스의 우선순위를 높여 먼저 작동하도록 배정해야 함

확장성

프로세스가 증가해도 시스템이 안정적으로 작동해야 함

반응 시간 보장

시스템은 적절한 시간 안에 프로세스의 요구에 반응해야 함

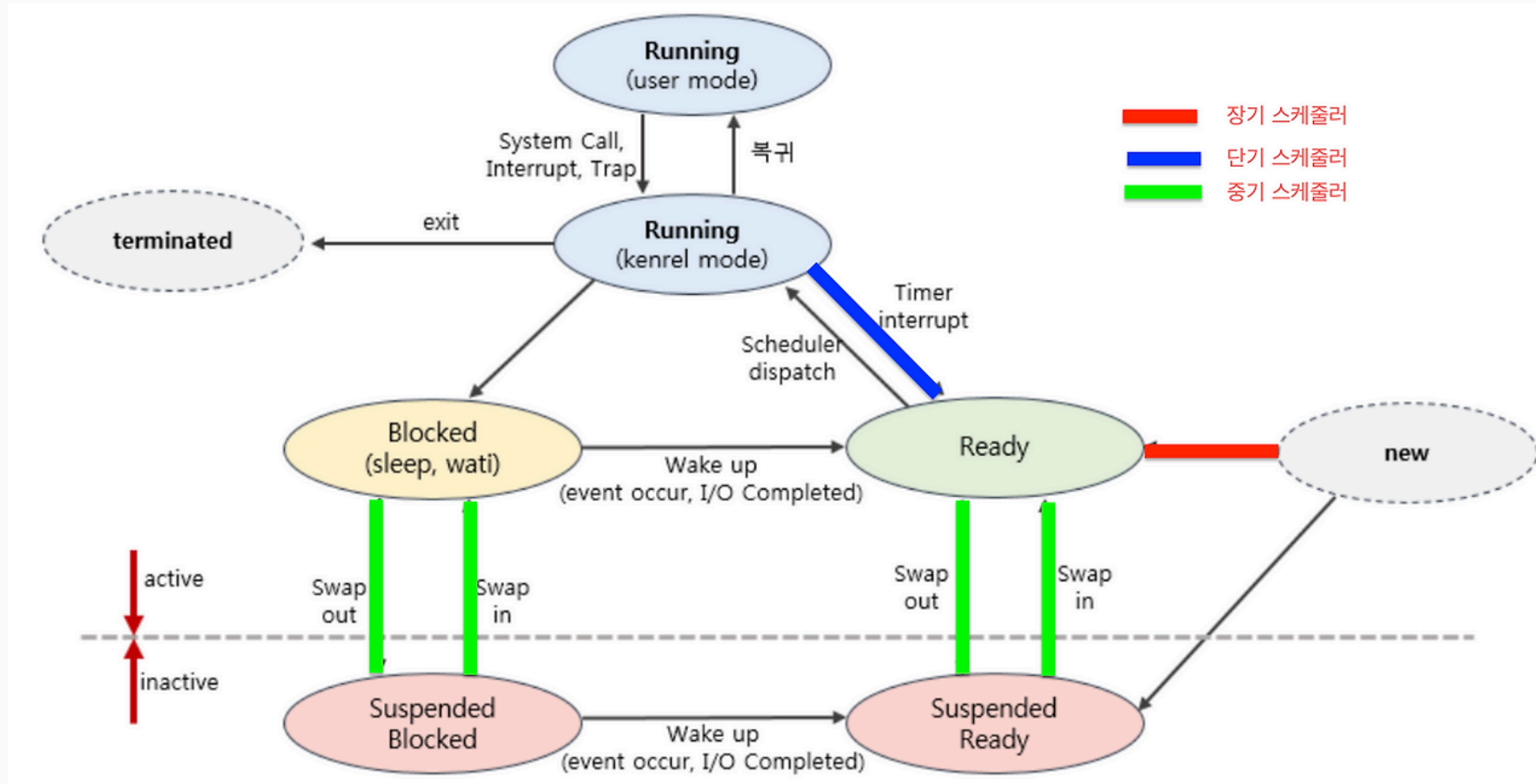
무한 연기 방지

특정 프로세스의 작업이 무한히 연기 되어서는 안 됨

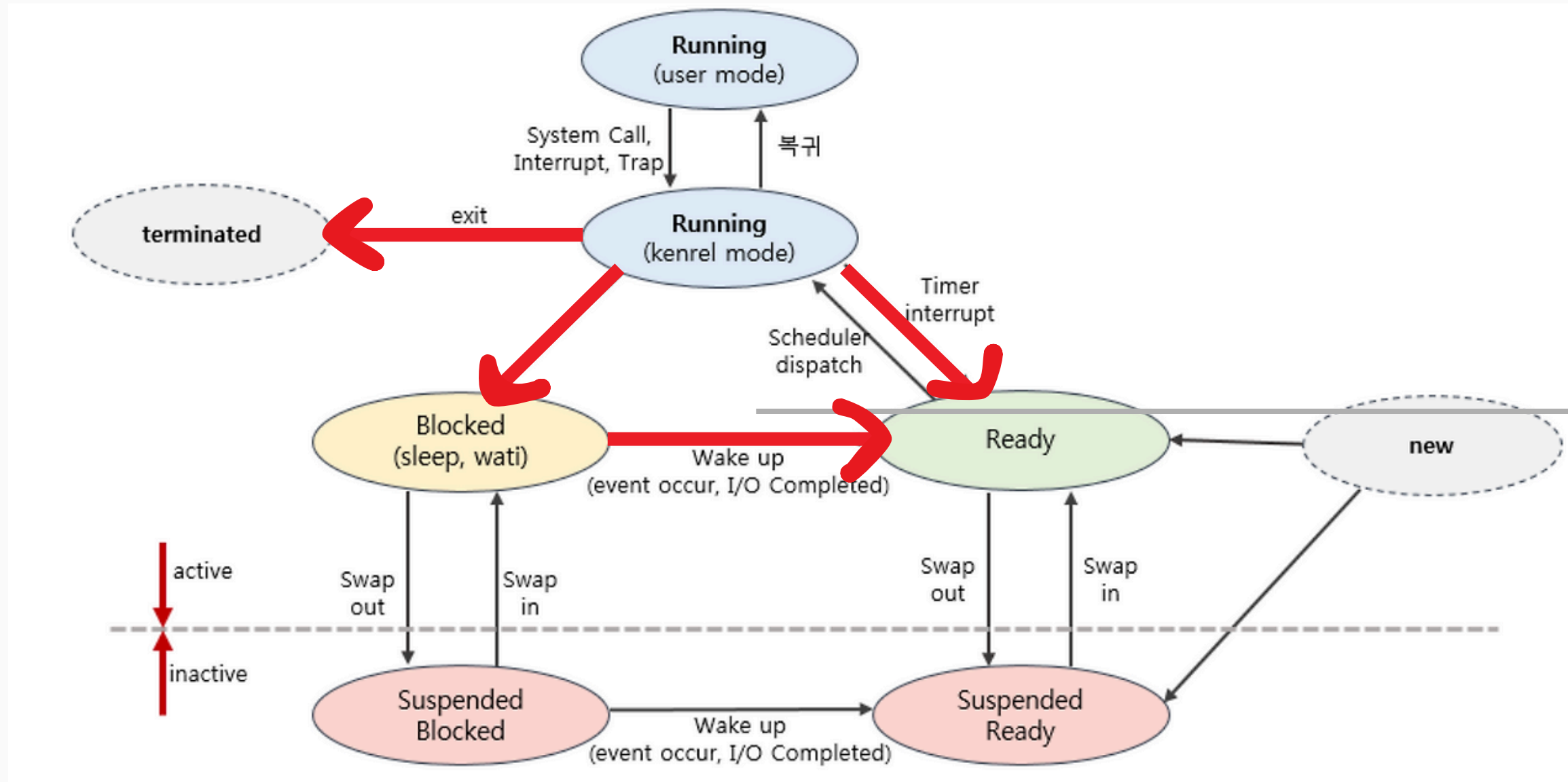
02

스케줄러의 계층적 구조

스케줄러의 계층적 구조



스케줄링 결정 시점



- 실행 상태 → 대기 상태
- 실행 상태 → 준비 상태
- 대기 상태 → 준비 상태
- 실행을 마치고 종료

03

선점형 vs 비선점형

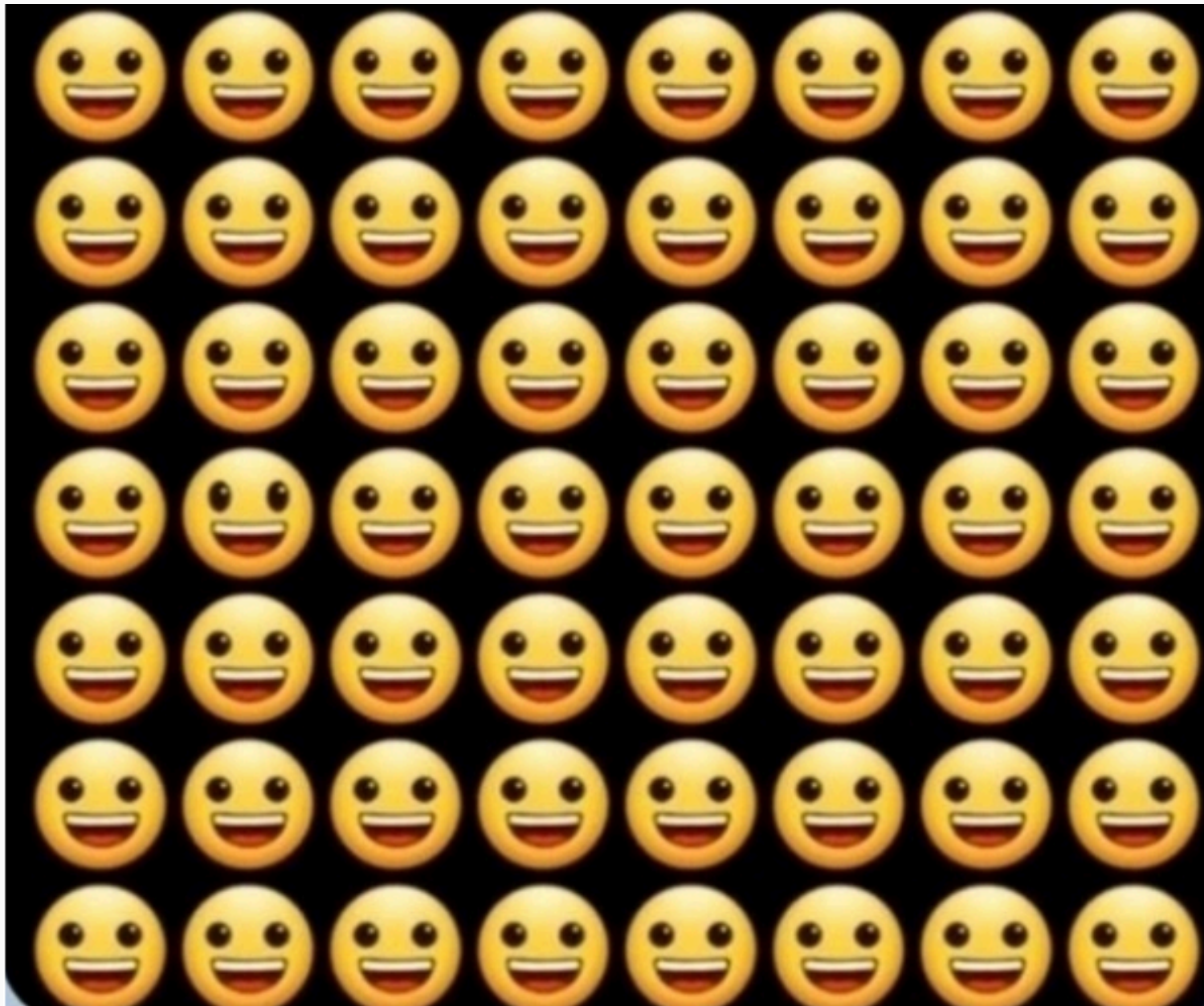
선점형 vs 비선점형



선점 (Preemptive)

- 한 프로세스에게 CPU가 할당되면 다른 프로세스가 CPU를 강제로 빼앗을 수 없는 방식
- 장점
 - 알고리즘이 단순하고 구현하기 쉬움
 - 문맥 교환으로 인한 오버헤드가 적음
 - 프로세스 순서가 정해져 있어 응답 시간 예측이 쉬움
- 단점
 - CPU 사용 시간이 긴 프로세스가 CPU를 독점하여 다른 프로세스가 무기한 대기하는 경우가 발생할 수 있음

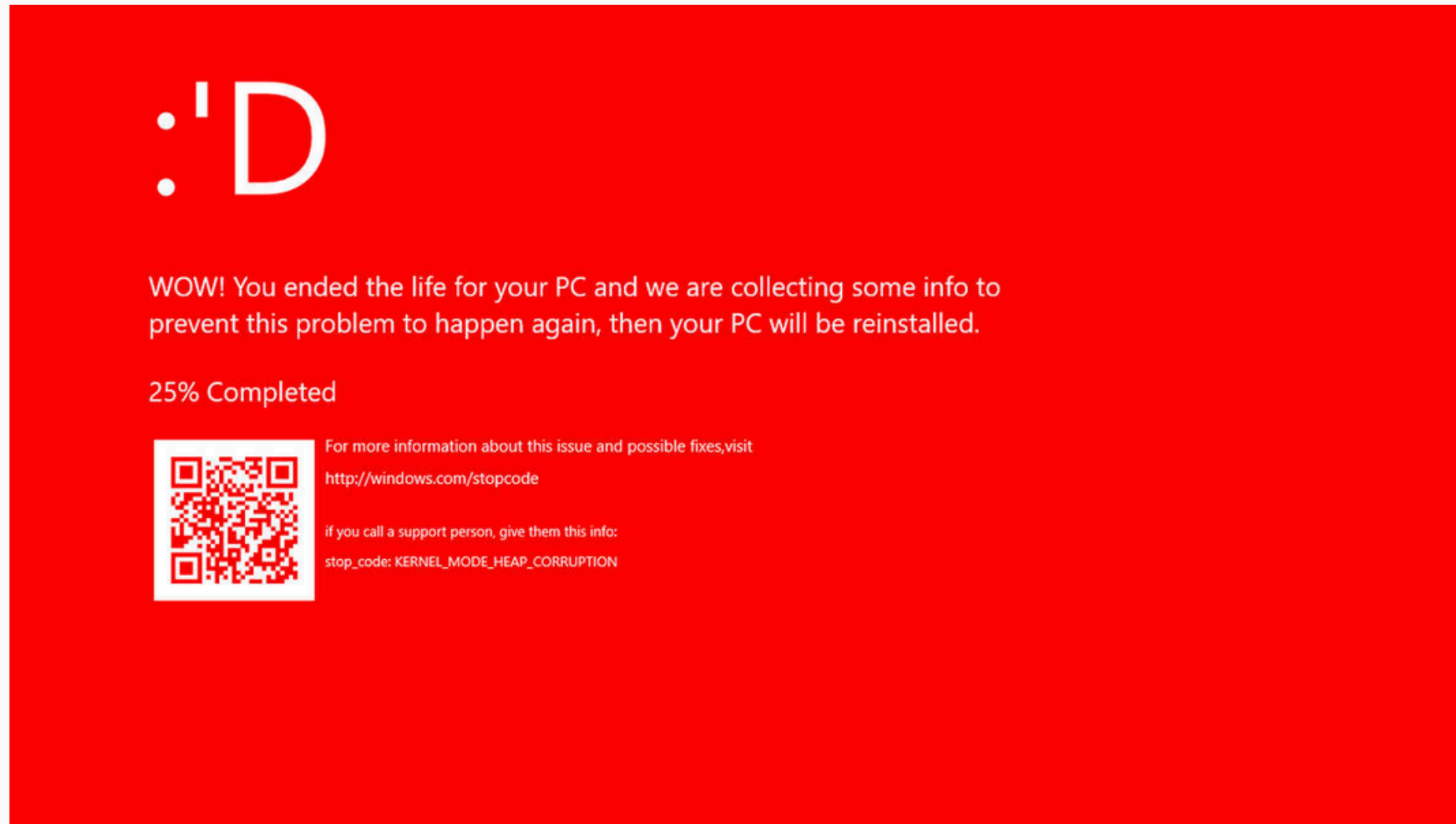
선점형 vs 비선점형



일괄 처리 시스템(Batch Processing System)

- 유사한 성격의 작업을 모아서 한 번에 순차적으로 처리하는 방식
- 장점
 - 사용자의 직접적인 개입 없이 자동으로 실행
- 단점
 - 같은 유형의 작업이 모일 때 까지 대기하는 방식
 - 작업이 끝날 때까지 대기해야 함

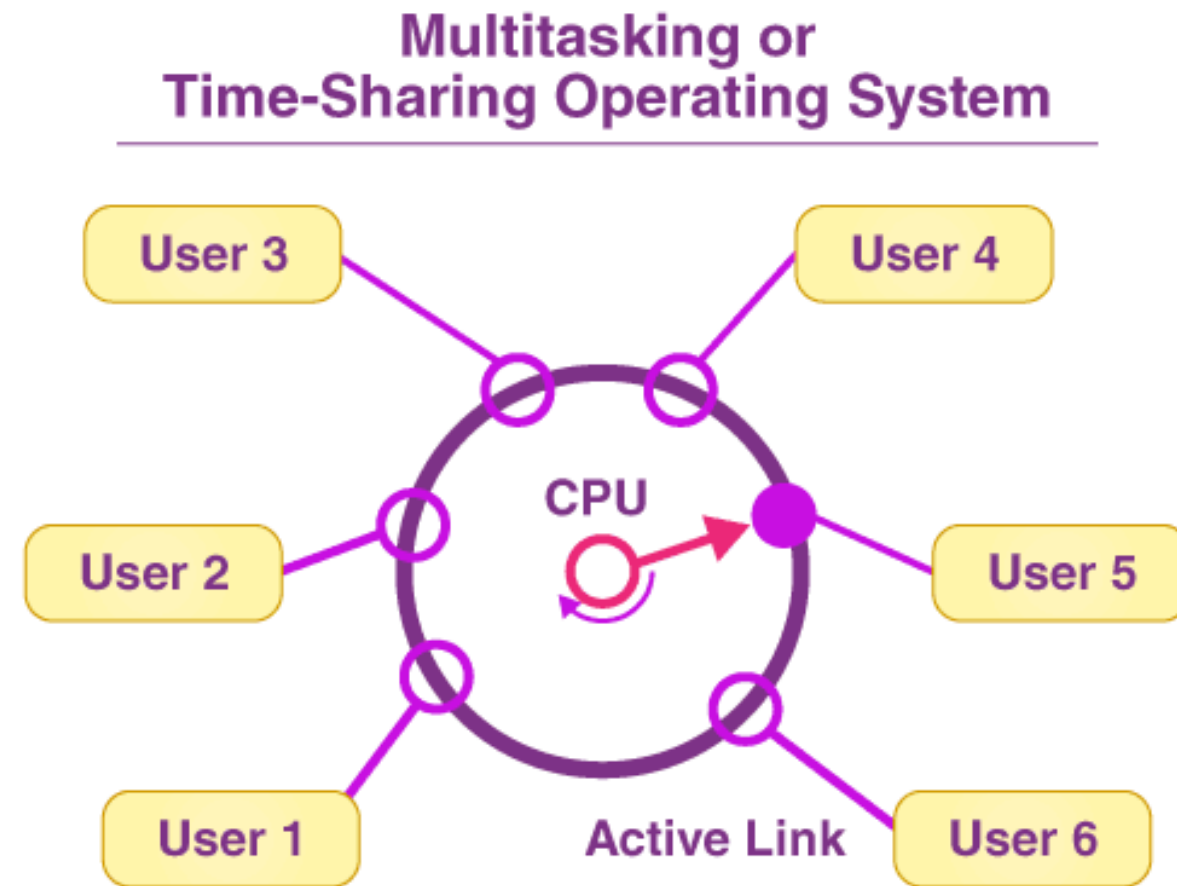
선점형 vs 비선점형



비선점 (Non-Preemptive)

- 현재 실행 중인 프로세스를 강제로 중단 시키고 CPU 제어권을 다른 프로세스에게 넘겨줄 수 있는 방식
- 장점
 - 긴급하거나 우선순위가 높은 프로세스가 CPU를 즉시 얻을 수 있음
 - 한 프로세스의 CPU 독점 방지 가능함
- 단점
 - 알고리즘이 복잡함
 - 잦은 문맥 교환으로 인한 오버헤드가 큼

선점형 vs 비선점형

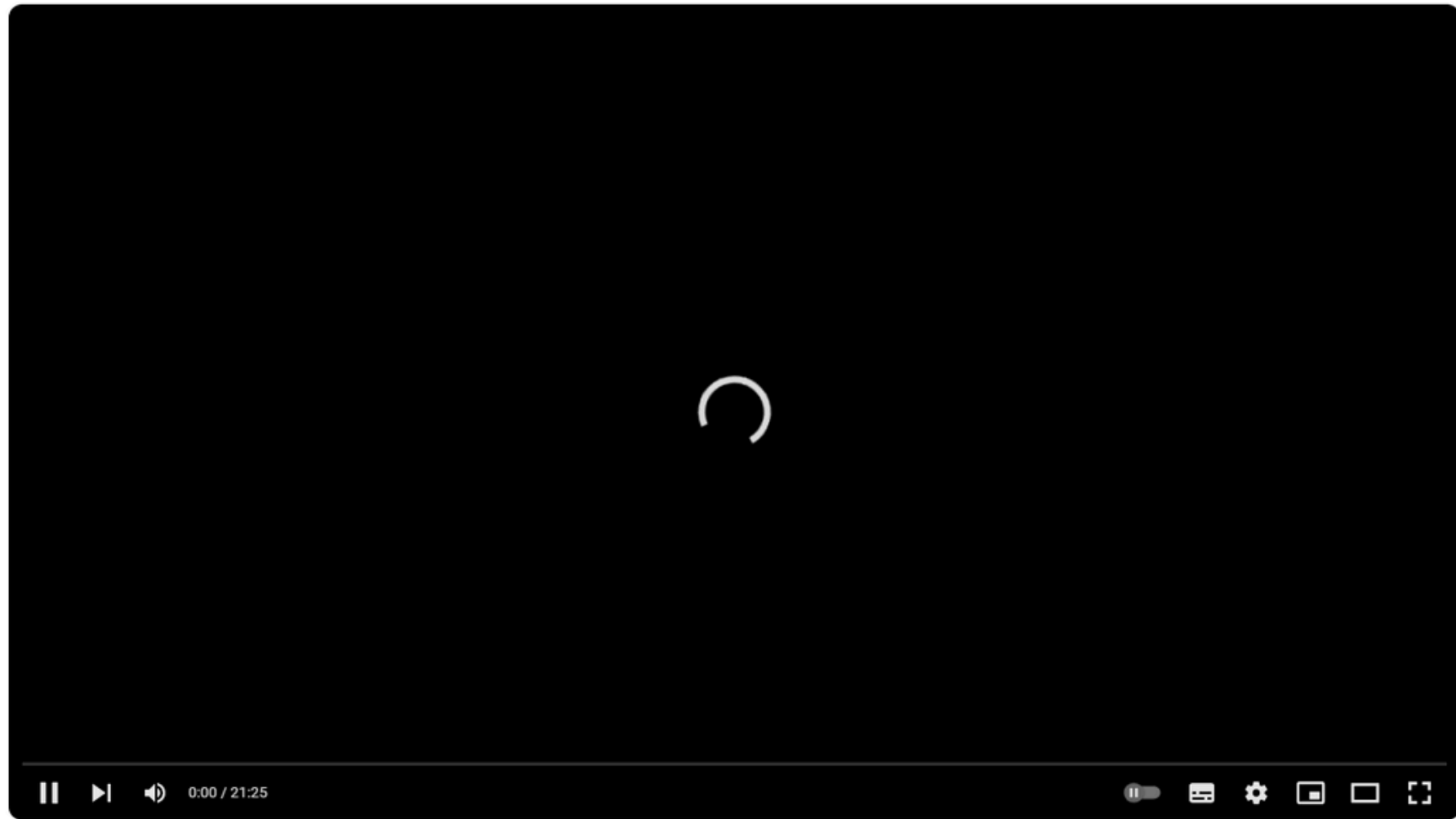


BYJU'S
The Learning App

시분할 시스템 (Time-Sharing System)

- 여러 작업을 짧은 시간 단위로 번갈아가면서 동시에 처리하는 방식
- 장점
 - CPU가 사용자 간 즉시 전환하여 응답 시간이 단축됨
 - CPU 유휴 시간을 최소화함
- 단점
 - 잦은 문맥 교환 비용이 발생함
 - 사용자 입장에서는 반응 속도가 느려질 수 있음

선점형 vs 비선점형



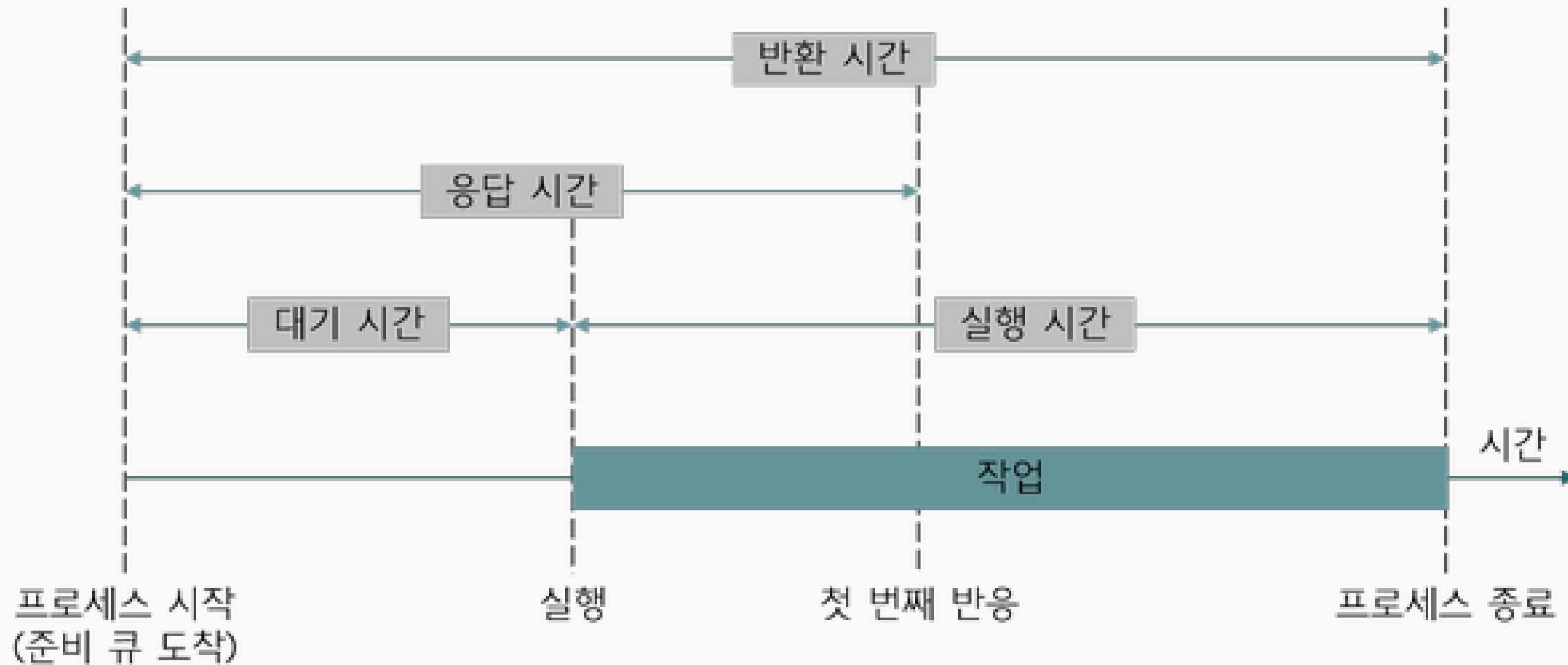
실시간 시스템(Real-Time System)

- 작업 처리에 제한 시간을 갖는 시스템
- 제한 시간 내 서비스를 제공하는 것이 자원 활용 효율보다 중요함
- Hard Real-Time Task
 - 시간 제약을 지키지 못하는 경우 치명적임
 - 발전소 제어, 무기 제어
- Soft Real-Time Task
 - 치명적인 문제는 일으키지 않으나 사용에 불편함
 - 동영상 재생

04

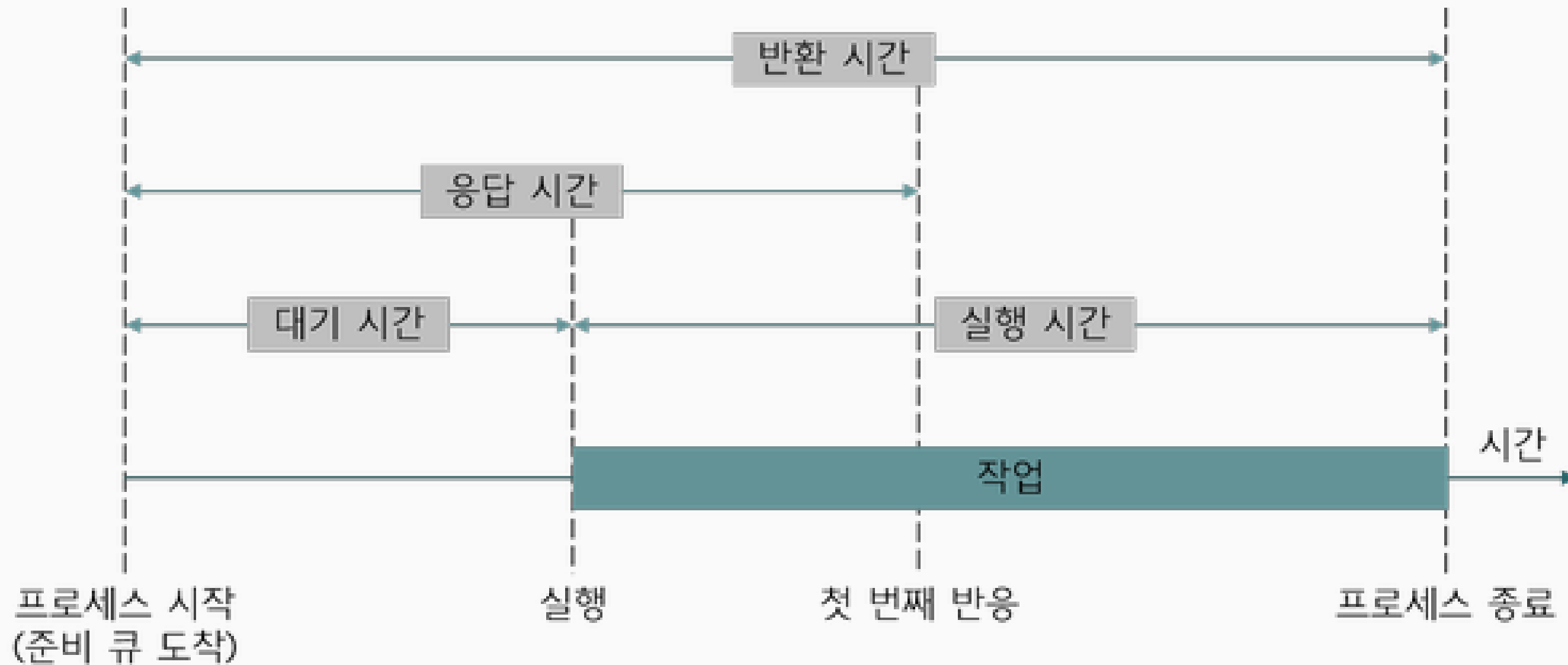
스케줄링 평가 기준

스케줄링 평가 기준



- CPU 이용률(CPU Utilization) : 전체 시간 중 CPU가 작업을 수행한 시간의 비율
- 처리량(Throughput) : CPU가 단위 시간동안 처리하는 프로세스의 개수

스케줄링 평가 기준



- 응답 시간(Response Time) : 프로세스가 시스템에 도착한 후 CPU를 할당받기까지 걸린 시간
- 대기 시간(Waiting Time) : 프로세스가 준비 큐에서 대기하는 시간의 총합
- 반환 시간(Turnaround Time) : 프로세스가 시작해서 끝날 때까지 걸리는 시간

05

스케줄링 알고리즘

스케줄링 알고리즘



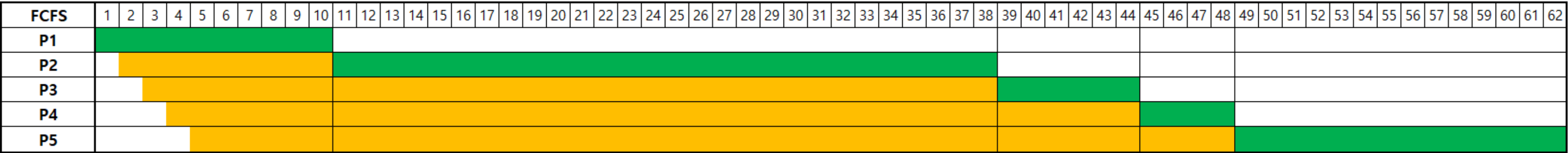
FCFS(First Come First Service)

- 준비 큐에 도착한 순서대로 CPU를 할당함
- 호위 효과(Convey Effect)
 - CPU 버스트 시간이 매우 긴 프로세스 하나가 짧은 여러 프로세스보다 먼저 도착하여 CPU를 점유하는 현상

스케줄링 알고리즘 (FCFS)

프로세스	도착 시간	실행 시간
P ₁	0	10
P ₂	1	28
P ₃	2	6
P ₄	3	4
P ₅	4	14

프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	10	10 - 0 = 10	10 - 10 = 0
P2	1	28	38	38 - 1 = 37	37 - 28 = 9
P3	2	6	44	44 - 2 = 42	42 - 6 = 36
P4	3	4	48	48 - 3 = 45	45 - 4 = 41
P5	4	14	62	62 - 4 = 58	58 - 14 = 44



- 평균 반환 시간 : 38.4
- 평균 대기 시간 : 26

스케줄링 알고리즘



SJF(Shortest-Job-First)

- 준비 큐에 있는 프로세스 중에서 다음 CPU 버스트 시간이 가장 짧은 프로세스에게 CPU를 할당함
- 기아 현상(Starvation)
 - CPU 버스트 시간이 짧은 프로세스들이 지속적으로 도착한다면, 대기 큐에 있는 버스트 시간이 긴 프로세스는 CPU를 영원히 할당 받지 못함

스케줄링 알고리즘 (SJF)

프로세스	도착 시간	실행 시간
P_1	0	10
P_2	1	28
P_3	2	6
P_4	3	4
P_5	4	14

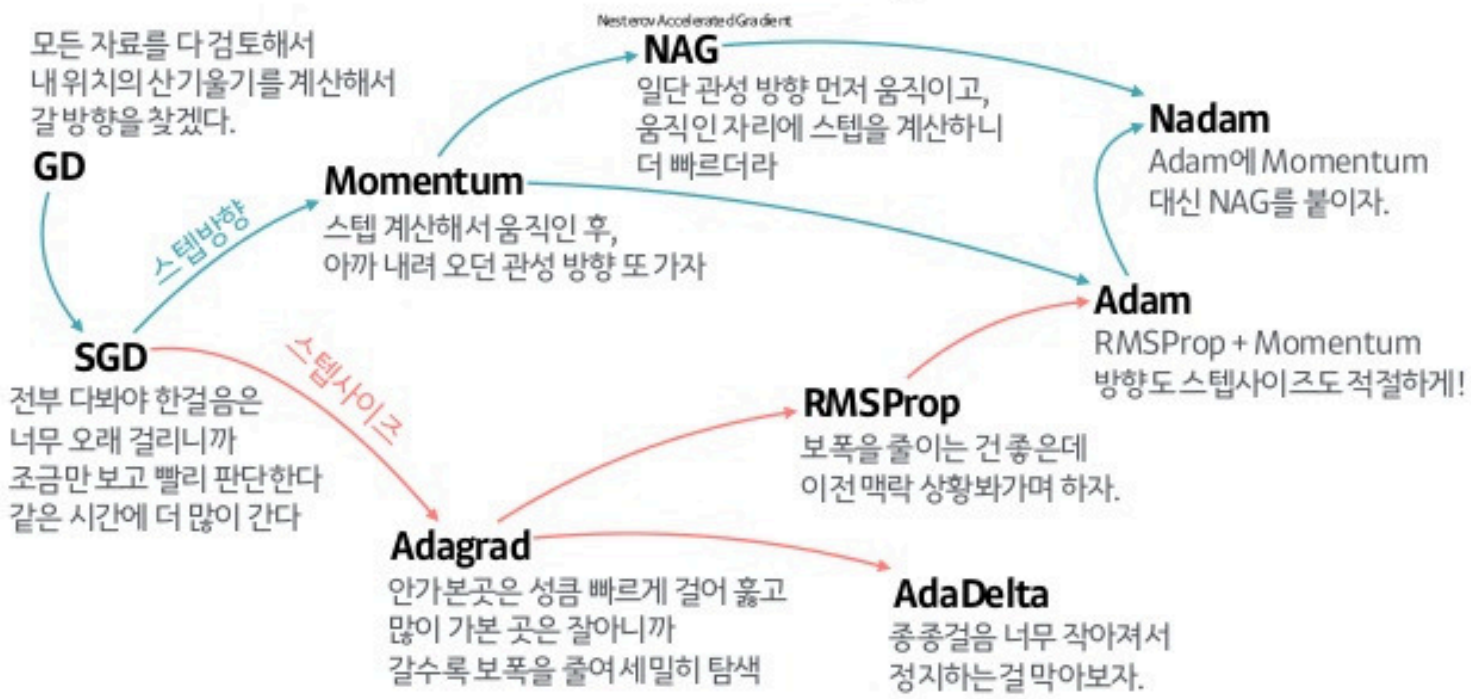
프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	10	$10 - 0 = 10$	$10 - 10 = 0$
P2	1	28	62	$62 - 1 = 61$	$61 - 28 = 33$
P3	2	6	20	$20 - 2 = 18$	$18 - 6 = 12$
P4	3	4	14	$14 - 3 = 11$	$11 - 4 = 7$
P5	4	14	34	$34 - 4 = 30$	$30 - 14 = 16$

SJF	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62
P1																																																														
P2																																																														
P3																																																														
P4																																																														
P5																																																														

- 평균 반환 시간 : 26
- 평균 대기 시간 : 13.6

스케줄링 알고리즘

산 내려오는 작은 오솔길 찾기(Optimizer)의 발달 계보



HRN(Highest Response Ratio Next)

- 대기 시간과 버스트 시간을 고려해서 다음에 사용할 프로세스를 결정함

$$\text{우선순위} = \frac{\text{대기 시간} + \text{CPU 사용 시간}}{\text{CPU 사용 시간}}$$

- 버스트 시간이 길더라도 언젠가는 실행이 보장됨

스케줄링 알고리즘 (HRN)

프로세스	도착 시간	실행 시간
P_1	0	10
P_2	1	28
P_3	2	6
P_4	3	4
P_5	4	14

프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	10	$10 - 0 = 10$	$10 - 10 = 0$
P2	1	28	62	$62 - 1 = 61$	$61 - 28 = 33$
P3	2	6	20	$20 - 2 = 18$	$18 - 6 = 12$
P4	3	4	14	$14 - 3 = 11$	$11 - 4 = 7$
P5	4	14	34	$34 - 4 = 30$	$30 - 14 = 16$

SJF	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62
P1																																																														
P2																																																														
P3																																																														
P4																																																														
P5																																																														

- 평균 반환 시간 : 26
- 평균 대기 시간 : 13.6

스케줄링 알고리즘



SRTF(Shortest Remaining Time First)

- 프로세스가 실행을 완료했을 때 남은 프로세스 중 가장 버스트가 짧은 것을 선택함
- SJF 알고리즘의 선점형 버전

스케줄링 알고리즘 (SRTF)

프로세스	도착 시간	실행 시간
P_1	0	10
P_2	1	28
P_3	2	6
P_4	3	4
P_5	4	14

프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	20	$20 - 0 = 20$	$20 - 10 = 10$
P2	1	28	62	$62 - 1 = 61$	$61 - 28 = 33$
P3	2	6	12	$12 - 2 = 10$	$10 - 6 = 4$
P4	3	4	7	$7 - 3 = 4$	$4 - 4 = 0$
P5	4	14	34	$34 - 4 = 30$	$30 - 14 = 16$

[illegible]

- 평균 반환 시간 : 25
- 평균 대기 시간 : 12.6

스케줄링 알고리즘



RR(Round Robin)

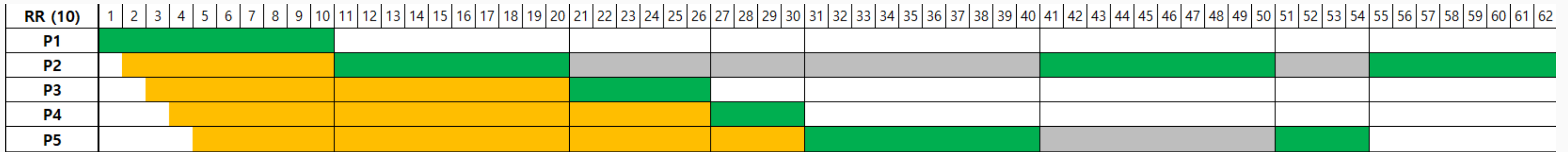
- FCFS + 시간 할당량(퀀텀)
- 퀀텀이 만료될 때 까지 프로세스가 실행 중이면 CPU는 프로세스를 강제로 선점시키고 준비 큐의 맨 뒤로 보냄
- ‘q’가 너무 큰 경우
 - FCFS처럼 동작
- ‘q’가 너무 작은 경우
 - 문맥 교환이 매우 빈번하게 발생함

스케줄링 알고리즘 (RR)

$$q = 10$$

프로세스	도착 시간	실행 시간
P_1	0	10
P_2	1	28
P_3	2	6
P_4	3	4
P_5	4	14

프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	10	$10 - 0 = 10$	$10 - 10 = 0$
P2	1	28	62	$62 - 1 = 61$	$61 - 28 = 33$
P3	2	6	26	$26 - 2 = 24$	$24 - 6 = 18$
P4	3	4	30	$30 - 3 = 27$	$27 - 4 = 23$
P5	4	14	54	$54 - 4 = 50$	$50 - 14 = 36$



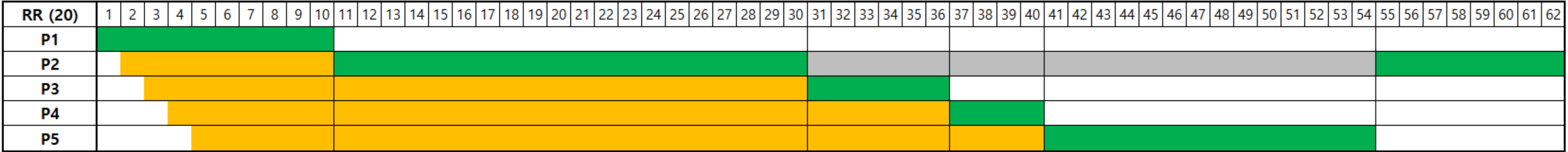
- 평균 반환 시간 : 34.4
- 평균 대기 시간 : 22

스케줄링 알고리즘 (RR)

q = 20

프로세스	도착 시간	실행 시간
P ₁	0	10
P ₂	1	28
P ₃	2	6
P ₄	3	4
P ₅	4	14

프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	10	10 - 0 = 10	10 - 10 = 0
P2	1	28	62	62 - 1 = 61	61 - 28 = 33
P3	2	6	36	36 - 2 = 34	34 - 6 = 28
P4	3	4	40	40 - 3 = 37	37 - 4 = 33
P5	4	14	54	54 - 4 = 50	50 - 14 = 36



- 평균 반환 시간 : 38.4
- 평균 대기 시간 : 26

스케줄링 알고리즘



우선순위(Priority)

- 각 프로세스에 우선순위를 부여하여 준비 큐에서 가장 높은 우선순위를 가진 프로세스가 실행됨
- 우선순위가 낮은 프로세스에게 기아 현상이 생길 수 있음
- 에이징(Aging)
 - 오랫동안 실행되지 못한 프로세스의 우선순위를 증가시킴

스케줄링 알고리즘 (Priority)

선점형

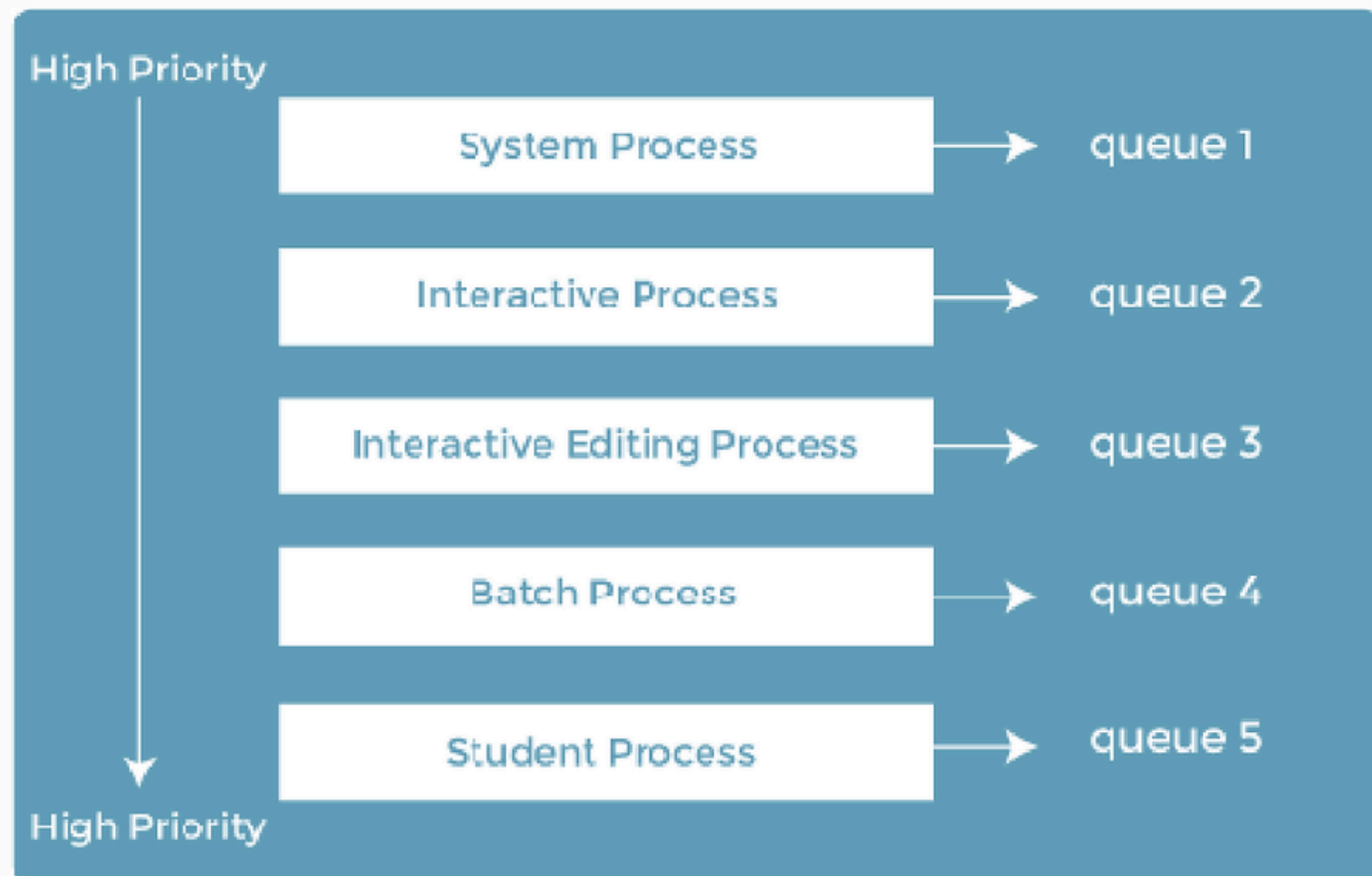
프로세스	도착 시간	실행 시간	우선순위
P ₁	0	10	3
P ₂	1	28	2
P ₃	2	6	4
P ₄	3	4	1
P ₅	4	14	2

프로세스	도착 시간	버스트 시간	종료 시간	반환 시간	대기 시간
P1	0	10	56	$56 - 0 = 56$	$56 - 10 = 46$
P2	1	28	33	$33 - 1 = 32$	$32 - 28 = 4$
P3	2	6	62	$62 - 2 = 60$	$60 - 6 = 54$
P4	3	4	7	$7 - 3 = 4$	$4 - 4 = 0$
P5	4	14	47	$47 - 4 = 42$	$42 - 14 = 29$

Priority (선)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62
P1																																																														
P2																																																														
P3																																																														
P4																																																														
P5																																																														

- 평균 반환 시간 : 39
- 평균 대기 시간 : 26.6

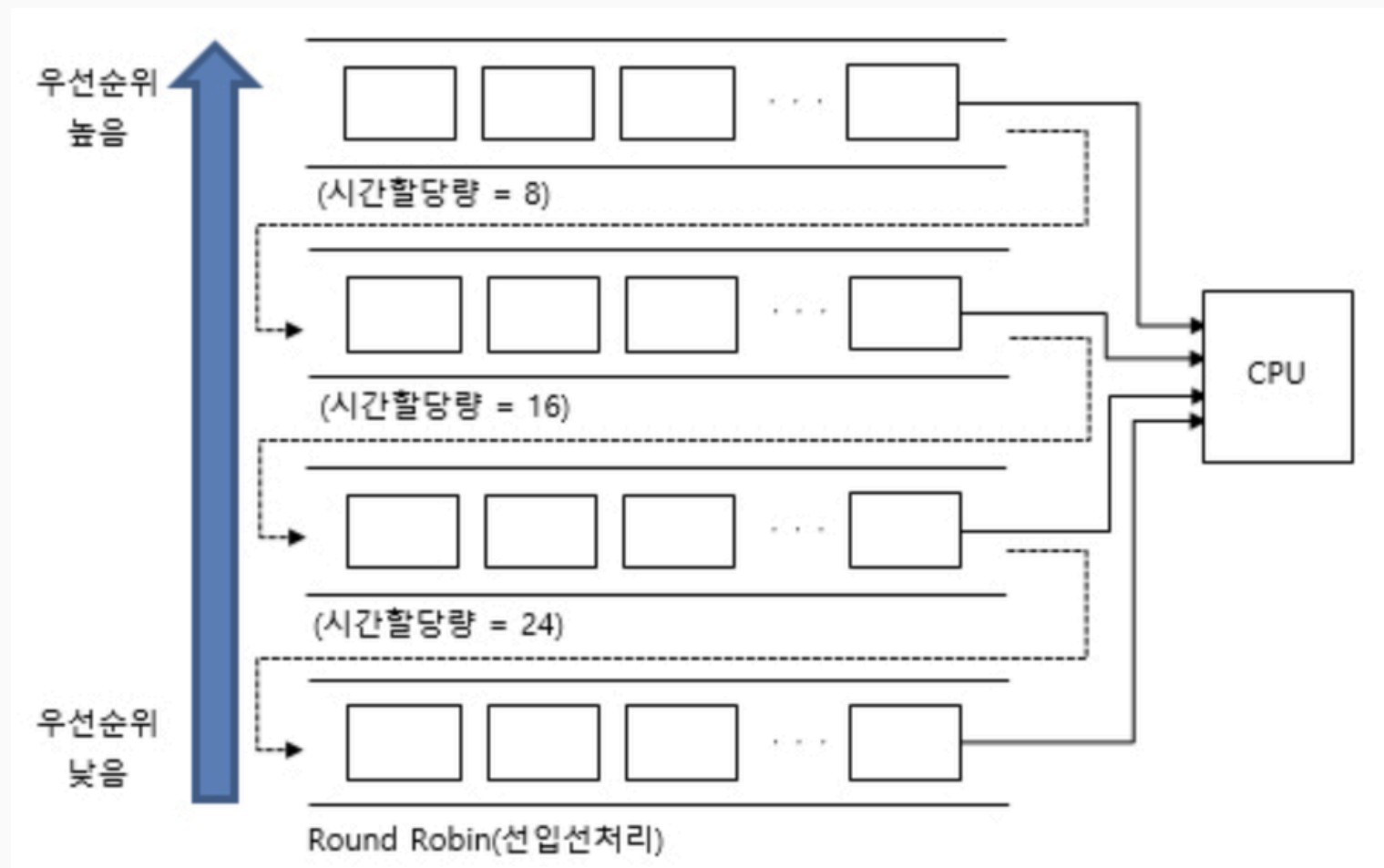
스케줄링 알고리즘



MLQ(Multi-Level Queue)

- 준비 큐를 여러 개의 독립적인 큐로 분할시키는 방식
- 각 큐는 자신만의 독립적인 스케줄링 알고리즘을 가짐
- 큐 사이에는 우선순위가 부여되어 상위 큐에 프로세스가 존재한다면 하위 큐의 프로세스는 CPU를 할당 받지 못함

스케줄링 알고리즘



MLFQ(Multi-Level Feedback Queue)

- 프로세스가 큐 사이를 이동할 수 있음
- 새로운 프로세스는 가장 높은 우선순위 큐에 들어감
- 타임 퀀텀을 다 사용하면 다음 우선순위 큐로 **강등**
- 다 사용하기 전에 I/O 요청 시 그대로 두거나 상위 큐로 **승격**
- 우선순위 부스팅
 - 가장 낮은 우선순위 큐에 있는 모든 프로세스를 가장 높은 우선순위 큐로 이동시킴
 - 무한정 대기 중인 작업 해결, 작업 도중 행동이 바뀐 프로세스에게 다시 높은 우선순위를 받을 기회를 제공

2025.11.14

감사합니다

operating system